

IFT436 - Série d'exercices #4 : les graphes

Manuel Lafond

Exercice 1: Donnez le code d'un algorithme qui trouve le chemin le plus court d'un sommet u à un sommet v sur un graphe *orienté*.

Solution.

Reprenez l'algorithme de fouille en largeur pour le cas non-orienté, et au lieu d'une boucle "pour chaque voisin z de w ", faites "pour chaque $z \in N^+(w)$ ", c'est-à-dire, assurez-vous d'aller vers les voisins sortants à chaque itération.

Exercice 2: Une stratégie naïve pour trouver le chemin le plus court entre deux sommets u et v est d'énumérer *tous* les chemins, et retourner le plus court.

C'est une mauvaise idée.

Pour le voir, montrez qu'il existe au moins $(n-2)!$ chemins entre deux sommets u et v dans un graphe complet.

Pour un peu plus de défi. Notez que le graphe complet a $\Theta(n^2)$ arêtes. Peut-être que sur les graphes avec peu d'arêtes, il y a peu de chemins. Montrez que c'est faux. C'est-à-dire, montrez qu'il existe des graphes avec $O(n)$ arêtes mais $\Omega(2^n)$ chemins entre deux sommets u et v .

Solution.

On suppose que G est un graphe complet et on va compter le nombre de chemins de longueur n . On sait qu'on démarre à u et qu'on termine à v . Il y a donc $n-2$ choix pour le premier sommet à visiter à partir de u , puis $n-3$ choix pour le prochain sommet, et ainsi de suite jusqu'à ce qu'il ne reste qu'un choix pour le prédécesseur de v (à chaque étape, tous les choix restants sont

possibles parce que le graphe est complet). Le nombre de chemins est donc $(n-2)(n-3)\dots(2)(1) = (n-2)!$.

Pour la deuxième partie, il faut faire attention de bien avoir $\Omega(2^n)$ sommets et non quelque chose du style $\Omega(2^{n/2})$, ce qui n'est pas la même chose. Considérez le graphe $G = (V, E)$ avec $V = \{a_1, a'_1, a_2, a'_2, \dots, a_k, a'_k\}$ et, pour chaque $i \in \{1, \dots, k-1\}$, les arêtes $a_i a'_i, a_i a_{i+1}, a_i a'_{i+1}, a'_i a_{i+1}$ et $a'_i a'_{i+1}$. Je vous invite à dessiner le graphe avec $k=3$ pour le visualiser.

Ce graphe a $n = 2k$ sommets et chaque sommet a au maximum 5 voisins. Donc le nombre d'arêtes est au maximum $10k \in \Omega(n)$, car ça diffère de n à une constante près. Comptons le nombre de façons de passer d'un sommet a_i à un sommet a_{i+1} ou bien a'_{i+1} , en supposant que a'_i n'a pas été visité. On peut soit passer de a_i à a_{i+1} ou a'_{i+1} directement, ou bien aller à a'_i pour ensuite aller à a_{i+1} ou a'_{i+1} . Il y a donc 4 façons. Le même argument s'applique si on veut passer de a'_i à la couche $i+1$. Maintenant, considérez le nombre de chemins qui démarrent de a_1 ou bien a'_1 et qui se rendent à a_k ou a'_k . Il y a 4 façons d'aller de la couche a_1 à a_2 , puis 4 façons d'aller de la couche a_2 à a_3 , et ainsi de suite. Le nombre de changements de couche est $k-1$, et donc ceci nous fait au moins $4^{k-1} = 4^k/4 = 4^{n/2}/4 = 2^n/4 \in \Omega(2^n)$.

Exercice 3: Le *diamètre* d'un graphe est la plus longue distance à parcourir pour aller d'un sommet à un autre (en supposant que l'on prend le chemin le plus court). Formellement, le diamètre s'écrit $diam(G)$ et se définit par

$$diam(G) = \max_{u,v \in V} dist(u, v)$$

où $dist(u, v)$ est la longueur du chemin le plus court entre u et v .

Donnez un algorithme en temps $O(n^2 + nm)$ qui trouve le diamètre d'un graphe.

Solution.

Il n'est pas difficile de modifier la fouille en largeur pour obtenir, pour $u \in V$, un des sommets à distance maximum de u . Il suffit de regarder quel sommet a été le dernier ajouté dans la file.

Pour trouver le diamètre, on fait une fouille en largeur pour chaque $u \in V$, et le diamètre sera le maximum de la distance entre un u et son sommet le plus éloigné. Puisqu'on fait n fouilles en largeur, ceci prendra un temps $O(n(n+m)) = O(n^2 + nm)$.

Exercice 4: Donnez un algorithme qui détermine si un graphe G est un arbre.

Solution.

Le plus simple est d'utiliser la définition disant que G est un arbre s'il est connexe et a $n - 1$ arêtes. L'algorithme va comme suit:

- si $m \neq n - 1$, return false
- si $getCC(G).size() > 1$, return false
- return true

La complexité est $O(n + m)$ pour $getCC$. Puisqu'à ce point, $m = n - 1$, on a $O(m + n) = O(n + n - 1) = O(n)$.

Exercice 5: Comparez la complexité des opérations “Supprimer une arête” et “Supprimer un sommet” sur une liste d'adjacence et sur une matrice d'adjacence. Notez que lorsqu'on supprime un sommet, toutes ses arêtes incidentes sont supprimées.

Solution.

Sur une liste d'adjacence, supprimer une arête uv demande de retirer un élément des deux listes pour u et v , ce qui demande un temps $O(deg(u) + deg(v))$. Pour supprimer un sommet v , il faut passer chaque liste dans laquelle v est présent. Ceci demande de parcourir chaque liste pour voir si v y est et, si oui, d'y faire une suppression. On peut optimiser un peu, mais il est difficile d'argumenter que c'est mieux que $O(n + \sum_{u \in V} deg(u)) \in O(n + m)$.

Sur une matrice d'adjacence, retirer une arête se fait en temps $O(1)$ mais retirer un sommet demande de recréer toute la matrice en temps $O(n^2)$.

Exercice 6: Démontrez les énoncés suivants sur les arbres.

- a. Montrez qu'un arbre avec au moins 2 sommets a au moins 2 feuilles.

- b. Montrez que si un arbre T a un sommet de degré k , alors T a au moins k feuilles.
- c. Montrez aussi que le seul arbre à n sommets et deux feuilles est un chemin de longueur $n - 1$.

Solution.

(a) — Preuve 1 —

Soit T un arbre avec au moins deux sommets. Considérons un chemin de longueur maximum dans T , que l'on dénote $P = (u, x_1, \dots, x_k, v)$ de T . Donc P va d'un sommet u à un autre sommet v . Notez que u ne peut pas avoir de voisin parmi $\{x_2, x_3, \dots, v\}$ car ceci impliquerait la présence d'un cycle (la seule exception est si $P = (u, v)$, dans quel cas T a deux sommets et donc deux feuilles). Notez aussi que u ne peut pas avoir de voisin à l'extérieur de $\{x_2, \dots, x_k, v\}$ car s'il y avait un tel voisin w , alors $P' = (w, u, x_1, \dots, x_k, v)$ serait un plus long chemin que P , contre-disant notre choix. Donc, u a un seul voisin, qui est x_1 . En utilisant les mêmes arguments, on déduit que v a aussi un seul voisin. Donc u et v sont des feuilles.

— Preuve 2 —

Voici une preuve par induction.

On montre l'énoncé par induction sur n . Cas de base: si $n = 2$, le seul arbre est $T = (\{u, v\}, \{uv\})$ et a effectivement deux feuilles. Induction: on suppose que tout arbre ayant moins de n sommets a au moins 2 feuilles. Soit $T = (V, E)$ un arbre, et soit $v \in V$ un sommet arbitraire. Considérez $T' = T[V \setminus \{v\}]$ l'arbre obtenu en retirant v et ses arêtes incidentes. Alors T' a un certain nombre de composantes connexes $T_1, \dots, T_k$, toutes des arbres car retirer un sommet ne peut pas créer de cycle.

Supposons d'abord que $k \geq 2$. Considérons T_i , $1 \leq i \leq k$. Si T_i a un seul sommet, alors ce sommet est une feuille dans T car son seul voisin est v . Si T_i a au moins deux sommets, par induction T_i a deux feuilles f_1 et f_2 . On sait que v n'a qu'un seul voisin dans T_i , sinon il y aurait un cycle dans T . Donc au moins un de f_1 ou f_2 est une feuille aussi dans T , car réinsérer v ne lui ajoute pas de voisin. On vient de montrer, en particulier, que T_1 a un sommet qui est une feuille dans T et T_2 aussi. Donc, T a au moins deux feuilles.

Supposons plutôt que $k = 1$. Par les mêmes arguments que ci-haut, T_1 a un sommet f qui est une feuille dans T . Par ailleurs, v est une feuille dans T puisqu'il a un seul voisin dans T_1 (et $k = 1$). Il s'ensuit que f et v sont deux feuilles de T .

(b)

Soit v un sommet de degré k avec les voisins $u_1, \dots, u_k$. Considérons $T' = T[V \setminus \{v\}]$. Soient $T_1, \dots, T_h$ les composantes connexes de T' . Il est facile de voir que pour tout u_i, u_j distincts, u_i et u_j ne sont pas dans la même composante: s'il y avait un chemin entre u_i et u_j dans T' , l'ajout de v à ce chemin créerait un cycle dans T .

Donc en fait, T' est constitué de k arbres $T_1, \dots, T_k$. Pour chacun de ces T_i , il y a deux possibilités. Si T_i n'est qu'un seul sommet u_i , alors u_i est une feuille dans T (son unique voisin est v). Sinon, T_i a au moins 2 feuilles. Au moins une de ces deux feuilles est aussi une feuille dans T . Il y a donc au moins k feuilles au total.

(c)

Si un arbre T n'est pas un chemin, il doit avoir un sommet de degré au moins 3, car la seule façon de n'avoir que des sommets de degré 1 et 2 est d'avoir un chemin (idéalement, il faudrait démontrer, peut-être par induction, mais nous ne nous étendrons pas).

Par le résultat précédent, T a donc au moins 3 feuilles. Donc tout arbre qui n'est pas un chemin a au moins 3 feuilles.

Exercice 7: Faites l'exercice 3.6 des notes de cours de Michael Blondin (trouver un chemin dans un grille numérotée).

Solution.

Il suffit de modéliser la grille comme un graphe et faire une fouille en largeur. Je vous laisse les détails.

Exercice 8: Soit $G = (V, E)$ un graphe et $X \subseteq V$. Le *sous-graphe induit* par X est dénoté $G[X]$ et est défini comme le graphe $(X, \{uv : uv \in E, u, v \in X\})$, c'est-à-dire, le graphe restreint à X et les arêtes à l'intérieur.

Étant donné X , combien de temps faut-il pour calculer $G[X]$? Quelles sont les suppositions à faire sur la façon de stocker G afin d'atteindre cette complexité?

Solution.

On peut procéder comme suit:

- Créer une graphe avec sommets X
- Pour chaque $u, v \in X$, ajouter uv ssi $uv \in E$.

Ceci prend un temps $O(|X|^2)$, si on suppose que vérifier si $uv \in E$ prend un temps constant.

Si X est grand, ceci peut être lent. Une autre approche est:

- Créer une graphe avec sommets X
- Pour chaque $v \in X$:
 - Pour chaque voisin w de v dans G :
 - si $w \in X$, ajouter vw

Ceci prend un temps $O(|X| + \sum_{v \in X} \deg_G(v)) = O(|X| + m)$.

Exercice 9: Soit $G = (V, E)$ un graphe. Une *composante connexe* de G est un sous-graphe induit $G[X]$, où $X \subseteq V$, tel que pour tout $u, v \in X$, il existe un chemin de u à v dans $G[X]$, et tel que X est maximal, c'est-à-dire, on ne peut pas ajouter de sommets à X en conservant cette propriété.

Décrivez un algorithme qui trouve toutes les composantes connexes de G et décrivez la complexité de votre algorithme.

Solution.

Voir les notes de cours.

Exercice 10: Soit G un graphe orienté. Une *composante fortement connexe* de G est un sous-graphe induit $G[X]$ maximal tel qu'il existe un chemin orienté entre toute paire de sommets $x, y \in X$ (ici, maximal veut dire qu'on ne peut pas ajouter de sommets en conservant cette propriété). Donnez un algorithme qui trouve toutes les composantes fortement connexes d'un graphe orienté.